Assignment Authoring Guide

This document explains how to create an assignment in Gradeforge and how to build tests using the direct builder.

It includes:

- a simple `I/O` example
- a simple `File I/O` example
- sample student programs for both examples
- student submission and run instructions
- professor workflow from assignment creation to review
- the exact fields to fill in the UI

1. Create the assignment

From the course workspace:

1. Open the course.
2. Go to `Assignments`.
3. Click `Create assignment`.
4. Fill in the assignment basics:
    - `Title`
    - `Description`
    - `Language`
    - `Due date`
    - `Max score`
    - allowed file types such as `.py` or `.java`
    - max attempts, if needed
5. Save the assignment.

Recommended setup:

- Python assignments:
    - language: `Python 3`
    - allowed file types: `.py`
- Java assignments:
    - language: `Java 17`
    - allowed file types: `.java`

2. Build the test suite

After the assignment is created:

1. Open the assignment.
2. Go to the `Tests` tab.
3. Click `Build tests`.
4. Choose one of:

    - `I/O`
    - `File I/O`

Common suite fields:

- `Suite name`: any staff-facing name
- `Visibility`:
    - `Private` means only staff see it
    - `Public` means students can see the suite version exists
- `Timeout (ms)`: optional per-suite timeout

3. When to use each mode

`I/O`

Use this when the program:

- reads from standard input
- prints to standard output
- does not need input/output files

Examples:

- calculator
- inventory console program
- menu-driven stdin/stdout assignment

`File I/O`

Use this when the program:

- takes command-line arguments
- reads input files
- writes output files
- needs expected output files to be checked

Examples:

- CSV processing
- report generation
- file transformation assignments

4. Important field rules

Python

- `I/O`:
    - no `Main class`
- `File I/O`:
    - requires `entry_path`
    - example: `main.py`

Java

- `I/O`:
  - requires `Main class`
  - example: `Main`
- `File I/O`:
  - requires `Main class`
  - example: `Program1`

Do not put:

- `java Main`
- `Main.java`
- `python main.py`

Only put the class name or entry path the builder expects.

5. Example A: Python `I/O` assignment

Assignment idea

Title:

```
Warehouse Inventory Console
```

Student program behavior:

- read a number `N`
- process `N` commands
- print the final inventory sorted by SKU
- print `TOTAL <sum>` at the end

Builder setup

In `Tests` -> `Build tests`:

- `Mode`: `I/O`
- `Language`: `Python 3`
- `Suite name`: `warehouse-io-tests`
- `Visibility`: `Private`
- `Timeout (ms)`: `3000`

Then add these cases.

Case 1

Input:

```

```

```
9
ADD A100 5
ADD B200 3
SHIP A100 2
RENAME B200 B250
SET C300 7
SHIP C300 8
ADD A100 4
DELETE B250
ADD D400 1
```

Expected output:

```
A100 7
C300 7
D400 1
TOTAL 15
```

Case 2

Input:

```
8
ADD CS1 10
ADD CS2 4
RENAME CS1 CS2
SHIP CS2 5
SET M1 0
ADD M1 2
DELETE ZZ9
SHIP M1 1
```

Expected output:

```
CS2 9
M1 1
TOTAL 10
```

Case 3

Input:

```
6
ADD X1 2
```

```
DELETE X1
ADD Y2 3
SHIP Y2 3
SET Z3 0
DELETE T1
```

Expected output:

```
EMPTY
TOTAL 0
```

What students submit

Students submit Python source files such as:

```
main.py
```

The tests only compare:

- stdin
- stdout

Sample student solution

Sample file:

- `docs/main.py`

This file implements the exact Python `I/O` assignment used in the example above.

Example program:

```python
def main():
    inventory = {}

    try:
        command_count = int(input().strip())
    except Exception:
        print("EMPTY")
        print("TOTAL 0")
        return

    for _ in range(command_count):
        try:
            parts = input().strip().split()
        except EOFError:
            break
```

```
        if not parts:
            continue

        command = parts[0]

        if command == "ADD":
            sku = parts[1]
            qty = int(parts[2])
            inventory[sku] = inventory.get(sku, 0) + qty

        elif command == "SHIP":
            sku = parts[1]
            qty = int(parts[2])
            current = inventory.get(sku, -1)
            if current >= qty:
                updated = current - qty
                if updated == 0:
                    inventory.pop(sku, None)
                else:
                    inventory[sku] = updated

        elif command == "SET":
            sku = parts[1]
            qty = int(parts[2])
            if qty <= 0:
                inventory.pop(sku, None)
            else:
                inventory[sku] = qty

        elif command == "DELETE":
            sku = parts[1]
            inventory.pop(sku, None)

        elif command == "RENAME":
            old_sku = parts[1]
            new_sku = parts[2]
            if old_sku in inventory:
                qty = inventory.pop(old_sku)
                inventory[new_sku] = inventory.get(new_sku, 0) + qty

    total = 0
    if not inventory:
        print("EMPTY")
    else:
        for sku in sorted(inventory):
            print(f"{sku} {inventory[sku]}")
            total += inventory[sku]

    print(f"TOTAL {total}")


if __name__ == "__main__":
    main()
```

How a student runs it locally

From the project root:

```
python3 docs/main.py
```

Then paste input such as:

```
9
ADD A100 5
ADD B200 3
SHIP A100 2
RENAME B200 B250
SET C300 7
SHIP C300 8
ADD A100 4
DELETE B250
ADD D400 1
```

Expected output:

```
A100 7
C300 7
D400 1
TOTAL 15
```

6. Example B: Java `File I/O` assignment

Assignment idea

Title:

```
Student Record File Processor
```

Student program behavior:

- reads input records from `input.csv`
- writes valid records to `valid.csv`
- writes invalid rows to `errors.txt`

Program runs like:

```
java Program1 input.csv valid.csv errors.txt
```

Builder setup

In `Tests` -> `Build tests`:

- `Mode`: `File I/O`
- `Language`: `Java 17`
- `Suite name`: `program1-file-tests`
- `Visibility`: `Private`
- `Timeout (ms)`: `5000`
- `Main class`: `Program1`

Case 1

Case name:

```
case-1
```

Command args:

```
input.csv
valid.csv
errors.txt
```

Input files:

`input.csv`

```
1001,Smith,Alice,CS
1002,Jones,Bob,IT
BAD_ID,Brown,Chris,CS
```

Expected files:

`valid.csv`

```
1001,Smith,Alice,CS
1002,Jones,Bob,IT
```

`errors.txt`

```
BAD_ID,Brown,Chris,CS
```

Case 2

Command args:

```
input.csv
valid.csv
errors.txt
```

Input files:

`input.csv`

```
2001,Lee,Anna,MATH
2002,Kim,John,ENG
```

Expected files:

`valid.csv`

```
2001,Lee,Anna,MATH
2002,Kim,John,ENG
```

`errors.txt`

```

```

What the builder does

During grading, the system:

1. creates `input.csv` in a temporary workspace
2. runs:

   ```
   java Program1 input.csv valid.csv errors.txt
   ```

3. checks the produced files against:
   - `valid.csv`
   - `errors.txt`

Sample student solution

Sample file:

- `docs/Program1.java`

This file implements the same Java `File I/O` assignment used in the example above.

Example program:

```
import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.util.ArrayList;
import java.util.List;

public class Program1 {
    public static void main(String[] args) throws IOException {
        if (args.length < 3) {
            System.err.println("Usage: java Program1 <input.csv>
<valid.csv> <errors.txt>");
            return;
        }

        Path inputPath = Paths.get(args[0]);
        Path validPath = Paths.get(args[1]);
        Path errorPath = Paths.get(args[2]);

        List<String> validRows = new ArrayList<>();
        List<String> errorRows = new ArrayList<>();

        try (BufferedReader reader = Files.newBufferedReader(inputPath)) {
            String line;
            while ((line = reader.readLine()) != null) {
                String trimmed = line.trim();
                if (trimmed.isEmpty()) {
                    continue;
                }

                String[] parts = line.split(",", -1);
                if (parts.length != 4) {
                    errorRows.add(line);
                    continue;
                }

                String id = parts[0].trim();
                String lastName = parts[1].trim();
                String firstName = parts[2].trim();
                String major = parts[3].trim();
```

```
                if (!isValidId(id) || lastName.isEmpty() ||
firstName.isEmpty() || major.isEmpty()) {
                    errorRows.add(line);
                    continue;
                }

                validRows.add(id + "," + lastName + "," + firstName + ","
+ major);
            }
        }

        writeLines(validPath, validRows);
        writeLines(errorPath, errorRows);
    }

    private static boolean isValidId(String value) {
        if (value.length() != 4) {
            return false;
        }
        for (int i = 0; i < value.length(); i++) {
            if (!Character.isDigit(value.charAt(i))) {
                return false;
            }
        }
        return true;
    }

    private static void writeLines(Path path, List<String> lines) throws
IOException {
        try (BufferedWriter writer = Files.newBufferedWriter(path)) {
            for (int i = 0; i < lines.size(); i++) {
                writer.write(lines.get(i));
                if (i < lines.size() - 1) {
                    writer.newLine();
                }
            }
        }
    }
}
```

How a student runs it locally

Create a sample `input.csv`:

```
1001,Smith,Alice,CS
1002,Jones,Bob,IT
BAD_ID,Brown,Chris,CS
```

Compile:

```
javac docs/Program1.java
```

Run:

```
cd docs
java Program1 input.csv valid.csv errors.txt
```

Then inspect:

```
cat valid.csv
cat errors.txt
```

Expected `valid.csv`:

```
1001,Smith,Alice,CS
1002,Jones,Bob,IT
```

Expected `errors.txt`:

```
BAD_ID,Brown,Chris,CS
```

7. Student workflow

Submit the assignment

For a student, the usual flow is:

1. Open the course.
2. Open the assignment.
3. Read the description, instructions, and rubric.
4. Prepare the required files:
    - Python example: `main.py`
    - Java example: `Program1.java`
5. Click `Submit`.
6. Upload the required source files.
7. Confirm the submission.

What happens after submission:

- the submission gets an attempt number
- the worker grades it against the active test suite

- the submission page shows:
  - verification result
  - per-case results
  - submitted files
  - instructor feedback, if present

Use the `Run` tab as a student

The `Run` tab is for temporary execution only.

Students can use it to:

- try custom stdin
- upload a stdin text file
- inspect stdout and stderr
- check behavior before resubmitting

It does not:

- create a new submission
- replace the official test result
- change the grade

For Python `I/O`:

1. Open the assignment.
2. Go to `Run`.
3. Select the submission attempt.
4. Enter stdin directly or upload a stdin file.
5. Click `Run`.
6. Read the execution console output.

For Java `File I/O`:

1. Open the assignment.
2. Go to `Run`.
3. Select the submission attempt.
4. Set command args if needed.
5. Add temporary input files.
6. Click `Run`.
7. Review stdout, stderr, and produced files.

View results as a student

After grading completes, students can open the submission review page and see:

- overall verification status
- per-case results
- input/output details when available
- submitted files
- grade and percent, if assigned

- instructor note
- rubric breakdown, if released
- rubric reference files, if attached

8. Professor workflow

Create and configure the assignment

1. Open the course.
2. Go to `Assignments`.
3. Click `Create assignment`.
4. Set:
    - title
    - language
    - max score
    - due date
    - allowed file types
5. Save the assignment.

Build the test suite

1. Open the assignment.
2. Go to `Tests`.
3. Click `Build tests`.
4. Choose:
    - `I/O` for stdin/stdout assignments
    - `File I/O` for file-based assignments
5. Fill the suite fields.
6. Add cases.
7. Save and make the suite active.

Recommended practice:

- create the suite as `Private` first
- submit one sample solution yourself
- confirm the verification page shows the expected case details

Check the assignment with a sample submission

For the Python example:

1. Build the `I/O` suite from this document.
2. Submit `docs/main.py`.
3. Open `Submissions`.
4. Click the sample row.
5. Confirm:
    - all cases pass
    - each case shows the configured input and expected output

For the Java example:

1. Build the `File I/O` suite from this document.
2. Submit `docs/Program1.java`.
3. Open `Submissions`.
4. Click the sample row.
5. Confirm:
    - expected files match
    - produced files preview correctly

Review and grade submissions

From the submission review page, professor or TA can:

- inspect verification results
- inspect each case in detail
- preview submitted files
- open the grading section
- assign or update the main grade
- add an instructor note
- review the rubric
- attach rubric reference files for information

Recommended review order:

1. check overall verification
2. inspect failing cases first
3. open submitted files
4. open grading
5. set the final grade
6. add concise feedback

Use the professor `Run` tab

The `Run` tab is useful when you want to inspect a saved submission without changing its official result.

Use it to:

- try a different stdin case
- upload a custom stdin file
- test temporary file fixtures
- inspect stdout, stderr, and produced files

Do not use it as the official grade source.

Official grading still comes from:

- the active test suite
- professor or TA grading on the submission page

9. Quick checklist before publishing

Before saving a test suite, check:

- the assignment language is correct
- Python `File I/O` has the correct `entry_path`
- Java suites use the correct `Main class`
- command args match the file names in the case
- expected output preserves exact spacing and newlines
- timeout is high enough for compile/run time

10. Common mistakes

Wrong Java main class

Wrong:

```
Main.java
```

Correct:

```
Main
```

Wrong file I/O args

Do not put the full command in one field.

Wrong:

```
java Program1 input.csv valid.csv errors.txt
```

Correct args list:

```
input.csv
valid.csv
errors.txt
```

Missing Python entry path in `File I/O`

For Python `File I/O`, you must provide:

```
main.py
```

or the real path to the submitted entry script.

11. Run tab behavior

The Run tab is for temporary execution only.

It does not:

- create a new submission
- change the grade
- replace the official grading run

It is useful for:

- trying custom stdin
- trying temporary file inputs
- checking program behavior before re-submitting

12. Recommended workflow

For most assignments:

1. Create the assignment.
2. Set the language and allowed file types.
3. Build a private test suite first.
4. Submit one sample solution yourself.
5. Open the submission review page and verify:
    - each case appears
    - expected input/output is correct
    - file previews look correct
6. If needed, publish a new suite version.